Speech Understanding

Assignment 1

End-to-End Speech Processing, Representation Learning & Ethical AI Pipeline

---

Submitted By

Purushothaman S
M25DE1033
M-Tech Data Engineering
Indian Institute of Technology Jodhpur

Github: [get the assignment from here]

Contents

Overview 3
System Architecture 3
Project Structure 4
Q1 -- Signal Processing & Feature Extraction 5
Manual MFCC Pipeline 5
Spectral Leakage & SNR Comparison 5
Voiced / Unvoiced Detection 6
Phonetic Boundary Mapping 6
Q2 -- Speaker Recognition & Representation Learning 7
Dataset & Configuration 7
Baseline Speaker Model 7
Reduced Disentangled Model 7
Training & Evaluation 8
Q3 -- Ethical AI Pipeline 8
Dataset Audit 8
Privacy-Preserving Transformation 8
Fairness-Aware Training 9
Proxy Evaluation Metrics 9
Setup & Reproduction 9
Option A -- Conda (Local) 9
Option B -- Docker (Recommended) 10
Data Requirements 10
Limitations & Future Directions 10
Summary 11
Appendix -- Key Dependencies 11

Overview

This assignment builds an end-to-end speech processing pipeline across three interconnected problems: manual feature extraction, speaker recognition with disentangled representation learning, and a responsible AI pipeline covering privacy and fairness. Each question builds on the insights of the previous one, forming a coherent research-grade system that is fully reproducible via Docker.

> **Pipeline at a Glance**
> Q1 extracts acoustic features from raw audio from scratch. Q2 trains and evaluates speaker-identification models, including a disentangled variant. Q3 audits the dataset for bias, applies a privacy transformation, and trains with a fairness regulariser -- then evaluates all steps with proxy metrics.

System Architecture

The diagram below shows how all modules connect, from raw audio input through to final outputs.

INPUT LAYER -- Raw WAV Audio Files (16 kHz, mono)

v main.py orchestrates all modules in sequence

Q1: SIGNAL PROCESSING & FEATURE EXTRACTION

MFCC Manual	Spectral Leakage & SNR	Voiced / Unvoiced	Phonetic Mapping

v Extracted features feed into representation learning

Q2: REPRESENTATION LEARNING -- SPEAKER RECOGNITION

Baseline Speaker Model	Disentangled Model
`FC: 16000 -> 512 -> 128 -> N speakers`	`Encoder(256) -> Speaker Head + Env Head`

v Embeddings reviewed under ethical AI lens

Q3: ETHICAL AI -- PRIVACY, FAIRNESS & AUDITING

Dataset Audit	Privacy Module	Fairness Training

v

OUTPUT -- Plots, Models, Transformed Audio, Metrics (outputs/)

Project Structure

The repository is organised into clearly separated modules for each question. A single entry point, main.py, orchestrates the full pipeline by executing each script in sequence:

File / Module	Purpose
main.py	Top-level orchestrator -- runs the entire Q1 -> Q2 -> Q3 pipeline in sequence
q1/mfcc_manual.py	Manual MFCC extraction -- pre-emphasis, framing, windowing, filterbank, DCT
q1/leakage_snr.py	Window comparison -- spectral leakage ratio and SNR across rectangular, Hamming, Hanning
q1/voiced_unvoiced.py	Cepstrum-based voiced/unvoiced segmentation with visual output
q1/phonetic_mapping.py	Wav2Vec2-driven phonetic boundary detection with RMSE against manual bounds
q2/train.py	Trains both the baseline and disentangled speaker models
q2/eval.py	Evaluates saved checkpoints, reports accuracy and confusion matrix
q2/configs/config.yaml	Centralised experiment configuration (dataset paths, hyperparameters, epochs)
q3/audit.py	Analyses dataset distributions for gender, age group and dialect imbalances
q3/privacymodule.py	Neural identity-obfuscation transformation module
q3/pp_demo.py	Runs the privacy transformation and saves original vs. transformed audio

q3/train_fair.py	Fairness-aware training with a group-loss-disparity regulariser
q3/evaluation_scripts/proxy_metrics.py	Measures proxy SNR between original and transformed audio

Q1 -- Signal Processing & Feature Extraction

The first question requires building a complete MFCC pipeline from the ground up -- no library shortcuts such as librosa.feature.mfcc are used. Every step from loading raw audio to producing log-Mel cepstral coefficients is implemented manually, giving full visibility into the mathematics underlying speech features.

Manual MFCC Pipeline

The pipeline in mfcc_manual.py walks through seven explicit stages:

Stage	Description
1. Load Audio	Read WAV using soundfile; convert stereo to mono by averaging channels
2. Pre-emphasis	Apply a first-order high-pass filter (alpha = 0.97) to boost high-frequency detail
3. Framing	Segment the signal into 25 ms frames with 10 ms hop; zero-pad the final frame
4. Windowing	Multiply each frame by a Hamming window (configurable) to reduce spectral leakage
5. Power Spectrum	Compute the magnitude-squared FFT with N = 512 points
6. Mel Filterbank	Apply 26 triangular filters spaced linearly on the Mel scale
7. Log + DCT	Take the log energy, then apply a Type-II DCT; keep the first 13 coefficients

The resulting 13-dimensional MFCC vectors per frame are saved alongside a waveform plot and an MFCC heatmap to outputs/q1/mfcc/.

Spectral Leakage & SNR Comparison

The leakage_snr.py module compares three window types on a 30 ms segment extracted 0.5 seconds into the signal:

Window Type	Characteristic

Rectangular (boxcar)	No tapering -- highest leakage, highest theoretical SNR within the main lobe
Hamming	Moderate side-lobe suppression -- good all-round trade-off
Hanning (Hann)	Stronger roll-off than Hamming -- lowest leakage at the cost of a wider main lobe

Spectral leakage is quantified as the ratio of energy outside the three-bin main lobe to total energy. SNR is estimated as the signal-to-noise ratio between the original segment and the windowed version. Spectrum plots for each window are written to outputs/q1/leakage_snr/.

Voiced / Unvoiced Detection

The voiced_unvoiced.py script classifies each 25 ms frame using real cepstrum analysis. The real cepstrum of a frame is the inverse FFT of the log-magnitude spectrum. Low-quefrency energy (bins 0-19) captures slow spectral envelope variation typical of voiced speech, while high-quefrency energy (bins 20-79) captures fast pitch-related periodicity.

A frame is labelled voiced when its high-to-low quefrency energy ratio exceeds the median ratio across all frames. The output is a colour-coded waveform plot (green = voiced, red = unvoiced) saved to outputs/q1/voiced_unvoiced/.

Phonetic Boundary Mapping

The phonetic_mapping.py module uses the pre-trained Wav2Vec2 model (facebook/wav2vec2-base-960h) to identify token-change boundaries in the audio. These model-derived boundaries are compared against uniformly spaced manual reference boundaries (one every 200 ms) using RMSE on the first ten aligned boundaries. Results, the full transcription, and both boundary sequences are written to outputs/q1/phonetic_mapping/mapping_results.txt.

> **Note on Wav2Vec2**
> The model download is approximately 378 MB and occurs automatically on first run. The script requires a 16 kHz WAV file in data/q1_audio/ and raises a descriptive error if the sample rate does not match.

Q2 -- Speaker Recognition & Representation Learning

The second question explores whether separating acoustic identity from environmental conditions during training leads to cleaner speaker representations. Two model architectures are compared -- a straightforward baseline and a disentangled variant -- trained and evaluated on the same three-speaker dataset.

Dataset & Configuration

Data lives in data/q2_audio/ (Audio1.wav, Audio2.wav, Audio3.wav) with labels in data/q2_metadata.csv. Each row records a filename, a speaker ID, and an environment ID. All hyperparameters are centralised in q2/configs/config.yaml:

Parameter	Value
sample_rate / max_length	16,000 Hz / 16,000 samples (1 second per clip)
batch_size	8
epochs	5
learning_rate	0.001 (Adam optimiser)
num_speakers	10
Embedding dimension	128 (shared encoder outputs 256)

Baseline Speaker Model

The baseline is a three-layer fully connected network that takes the raw 16,000-sample waveform as input and maps it directly to speaker logits. Training minimises cross-entropy on speaker labels alone, with no awareness of environmental conditions.

```
Architecture:  Input(16000) -> Linear(512) -> ReLU -> Linear(128) -> ReLU ->
Linear(N speakers)
```

Reduced Disentangled Model

The disentangled model introduces a shared encoder that projects the waveform to a 256-dimensional embedding. Two task-specific heads branch from this bottleneck: one for speaker classification and one for environment classification. The loss function explicitly rewards speaker discrimination while penalising environment predictability:

```
Loss = CrossEntropy(speaker_labels) - lambda_env * CrossEntropy(environment_labels)
```

With lambda_env = 0.3, the encoder is encouraged to learn representations that are informative about speaker identity yet blind to recording environment -- a lightweight form of adversarial disentanglement achieved without a separate discriminator network.

Training & Evaluation

Both models are trained simultaneously in train.py over five epochs. After training, eval.py loads the saved checkpoints, reports speaker identification accuracy for each model, and generates a confusion matrix for the disentangled model. Outputs land in outputs/q2/results/ and outputs/q2/models/.

Q3 -- Ethical AI Pipeline

The third question treats the speech system as a product that will affect real people and asks what obligations that creates. Three sub-tasks address dataset transparency, speaker privacy, and demographic fairness.

Dataset Audit

The audit.py script performs a systematic inspection of the dataset described in data/q3_metadata.csv. It analyses the distribution of audio samples across three demographic axes -- gender, age group (young / adult) and dialect (standard / regional) -- and flags missing or unreadable entries.

For a healthy speech recognition system these distributions should be as balanced as possible. A dataset skewed toward one gender, age group, or dialect will produce a model that under-performs for under-represented groups. The audit makes this visible before any model training happens.

Privacy-Preserving Transformation

The privacymodule.py defines a lightweight neural transformation that accepts a raw waveform and returns a modified waveform designed to obscure speaker identity. The architecture is a compact autoencoder-style block:

```
Input(16000) -> Linear(512) -> ReLU -> Linear(16000) -> Tanh
```

The pp_demo.py script loads the first WAV file from data/q3_audio/, runs it through the privacy module, and writes both the original and transformed audio to outputs/q3/pp_demo/. This makes it easy to listen to and compare the two versions side by side.

> **Scope Note**
> The current implementation is a proof-of-concept neural transformation, not a production voice anonymisation system. More sophisticated approaches -- formant shifting, voice conversion models, or differentially private feature extraction -- are listed as future improvements.

Fairness-Aware Training

The train_fair.py script introduces a group-equity regulariser on top of the standard classification loss. After computing per-sample cross-entropy losses, the fairness penalty is the difference between the highest and lowest average loss across demographic groups:

```
fairness_penalty = max(group_mean_losses) - min(group_mean_losses)
total_loss       = task_loss + 0.5 * fairness_penalty
```

This pushes the model toward equal per-group performance, narrowing the gap between how well it handles different demographic cohorts. The trained model is saved to outputs/q3/fair_training/fair_model.pt.

Proxy Evaluation Metrics

Because a full perceptual quality metric such as DNSMOS or FAD requires additional infrastructure, the proxy_metrics.py script computes a proxy signal-to-noise ratio between the original and privacy-transformed audio:

```
Proxy SNR (dB) = 10 * log10( signal_power / noise_power )
```

A higher proxy SNR means the transformation has introduced less distortion relative to the original signal power. A lower value means the transformation is more aggressive -- which may strengthen privacy protection but could degrade speech intelligibility.

Setup & Reproduction

The project ships with two equally supported ways to run the pipeline.

Option A -- Conda (Local)

```
conda create -n speech_assign python=3.10 -y
conda activate speech_assign
pip install -r requirements.txt
python main.py
```

Option B -- Docker (Recommended)

Docker ensures a fully reproducible environment regardless of host OS. The Dockerfile installs all dependencies including PyTorch, SoundFile, Transformers and scikit-learn.

```
docker build --no-cache -t speech-assignment .
docker run --rm -v ${PWD}:/app speech-assignment
```

Data Requirements

Question	Required Data
Q1	At least one 16 kHz mono WAV file in data/q1_audio/
Q2	Audio1.wav, Audio2.wav, Audio3.wav in data/q2_audio/ with matching data/q2_metadata.csv
Q3	Audio1.wav, Audio2.wav, Audio3.wav in data/q3_audio/ with data/q3_metadata.csv and data/q3_train.csv

All CSV files must be UTF-8 encoded and filenames must match exactly -- the system is case-sensitive inside Docker.

Limitations & Future Directions

The current system is deliberately scoped as an introductory yet extensible research pipeline. Several directions would significantly strengthen each component:

Area	Improvement
Larger datasets	Replace the three-file toy dataset with LibriSpeech or Mozilla Common Voice for meaningful generalisation
Richer evaluation	Swap proxy SNR for DNSMOS or Frechet Audio Distance for perceptual quality assessment
GPU acceleration	Add CUDA-aware data loaders and mixed-precision training for scalability
Better disentanglement	Explore variational autoencoders or adversarial discriminators for more principled representation separation
Real voice anonymisation	Implement formant shifting or a dedicated voice conversion model in place of the MLP proxy

Continuous fairness monitoring	Extend audit.py to run post-training and flag per-epoch equity degradation

Summary

This assignment delivers a complete, modular, and reproducible speech processing pipeline spanning signal processing, deep learning, and responsible AI. Starting from raw WAV files, the system manually extracts MFCCs, compares windowing strategies, detects voiced regions via cepstrum analysis, and maps phonetic boundaries using Wav2Vec2. A disentangled speaker model then learns to separate identity from environment, and the ethical AI pipeline closes the loop by auditing demographic balance, protecting speaker privacy through neural transformation, and enforcing cross-group fairness during training.

Each question can be run independently or triggered as a single end-to-end job through main.py, making the work easy to inspect, extend, and reproduce.

Appendix -- Key Dependencies

Package	Role
Python 3.10	Base runtime environment
PyTorch	Core deep learning framework for Q2 and Q3 models
soundfile	Reading and writing WAV audio files
numpy / scipy	Numerical processing -- FFT, windowing, DCT, framing
transformers (HuggingFace)	Wav2Vec2 processor and CTC model for phonetic mapping
scikit-learn	Accuracy scoring and confusion matrix for Q2 evaluation
matplotlib	Waveform, MFCC and segmentation plots
pandas	CSV input/output for metadata and result logging
PyYAML	Config file parsing for Q2 experiments
Docker	Reproducible containerised execution environment